QUANTUM SCIENCE CENTER

July 24th, 2026, OpenQSE Workshop (QCUF 2026)

QSC-CT
Fault Tolerant Quantum Computing Compiler and Q-IRIS runtime

Narasinga Rao Miniskar, Elaine Wong, Seyong Lee, Leyton Ortega Vicente, Jeffrey S Vetter, Travis Humble

miniskarnr@ornl.gov

Oak Ridge National Laboratory

U.S. DEPARTMENT of ENERGY

ORNL IS MANAGED BY UT-BATTELLE LLC
FOR THE US DEPARTMENT OF ENERGY

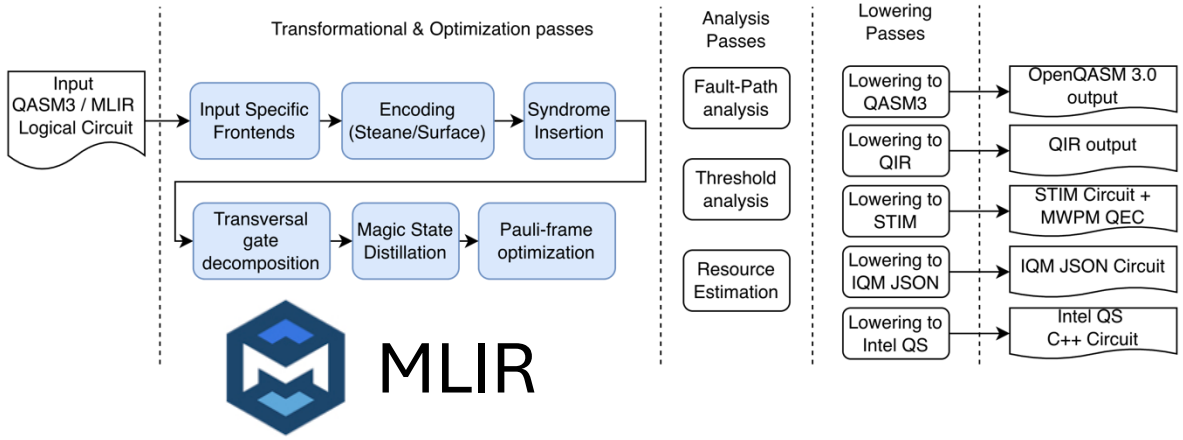
FTQC: An MLIR Dialect for Scalable Fault-Tolerant Quantum Computing

Narasinga Rao Miniskar, Elaine Wong, Seyong Lee, Vicente Layton Ortega, Yufei Ding, Jeff Vetter, Travis Humble

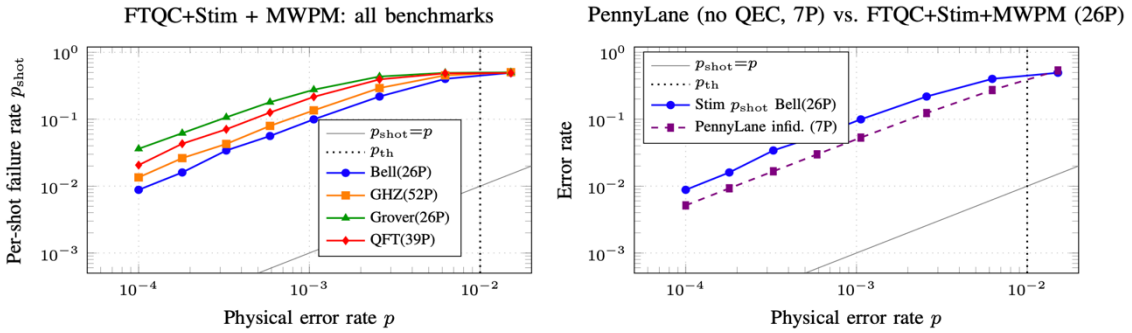
Objective

- Introduce FTQC, an open-source Multi-Level Intermediate Representation (MLIR) dialect and its framework.
- Provides a dedicated compilation pipeline for FTQC by treating error-correcting codes (ECC), syndrome extraction, and magic-state distillation as key intermediate representation (IR) constructs.

A Type-Safe MLIR Compiler Pipeline for Fault-Tolerant Quantum Computing



Logical Error Rates for FTQC+Stim+MWPM vs. Pennylane DM w/o QEC



Operation Categories and Counts

Category	Ops
Qubit lifecycle (init, encode, decode, dealloc)	5
Clifford gates (single- and two-qubit)	7
Non-Clifford gates (T, Toffoli, magic prep)	4
Syndrome extraction	5
Error correction & measurement	4
Pauli-frame management	3
Distillation & teleportation	3
Total	**31**

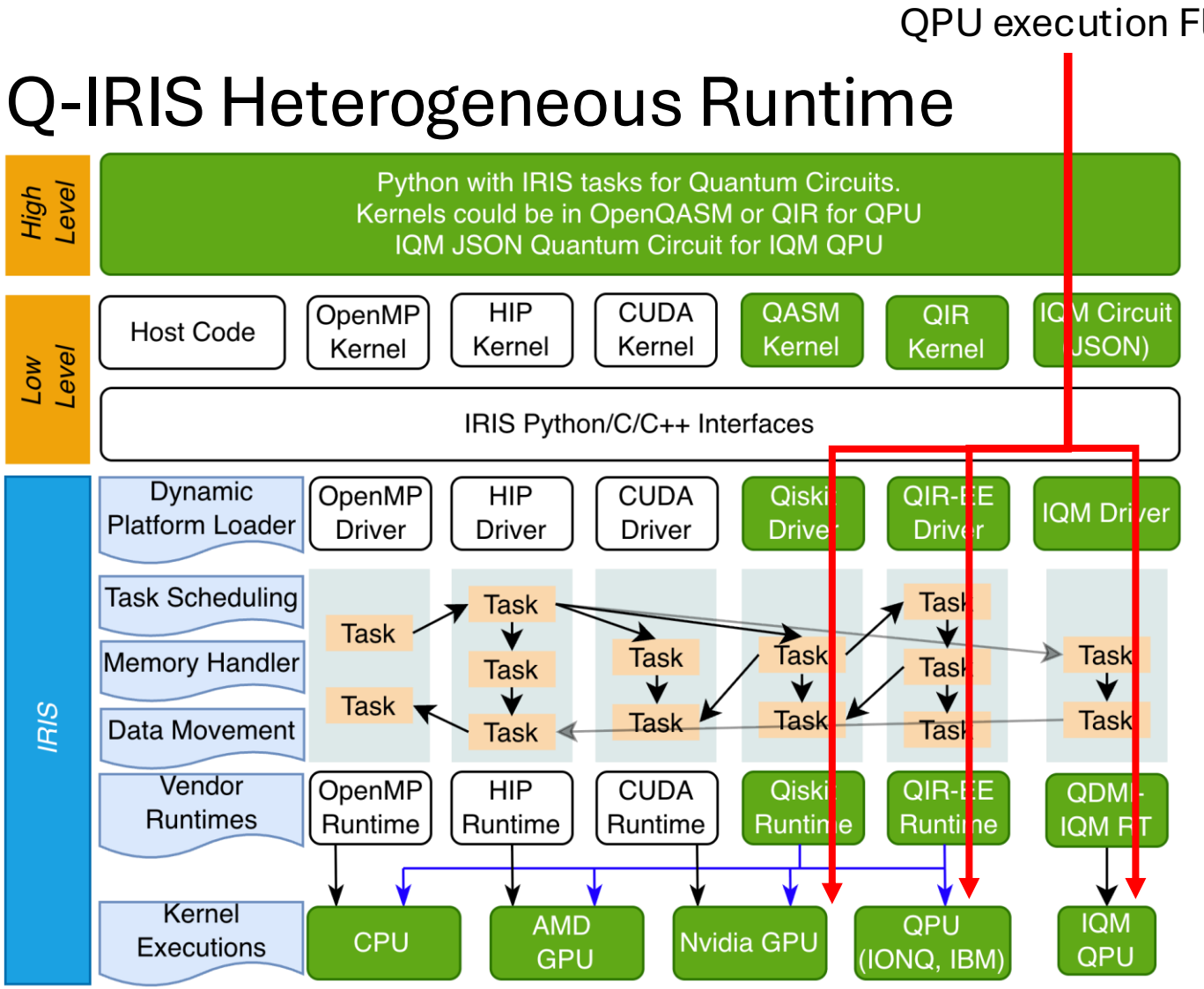
Example IR: Bell-state Preparation

```
func.func @bell_state_ft() -> (i1, i1) {
  %lq0 = ftqc.init_zero : <<steane, 3>>
  %lq1 = ftqc.init_zero : <<steane, 3>>
  %h0  = ftqc.logical_h %lq0 : <<steane, 3>>
  %c, %t = ftqc.logical_cnot %h0, %lq1
      : (<<steane, 3>>, <<steane, 3>>)
      -> (<<steane, 3>>, <<steane, 3>>)
  %b0 = ftqc.logical_measure %c
      : (<<steane, 3>>) -> i1
  %b1 = ftqc.logical_measure %t
      : (<<steane, 3>>) -> i1
  func.return %b0, %b1 : i1, i1
}
```

- MLIR can act as an exchange format for different programming models (QASM3, Qiskit, Pennylane QML) and can be made inter-operable with IRs (QIR, Qiskit IRs, Quake, HUGR,..), and lower to vendor-specific IR/programming model requirements
- Enables to write vendor agnostic passes
- WIP: Hardware aware FTQC and Integrate with Decoder

Q-IRIS Heterogeneous Runtime

https://github.com/ORNL/iris.git
https://code.ornl.gov/mach-q/iris (IQM)



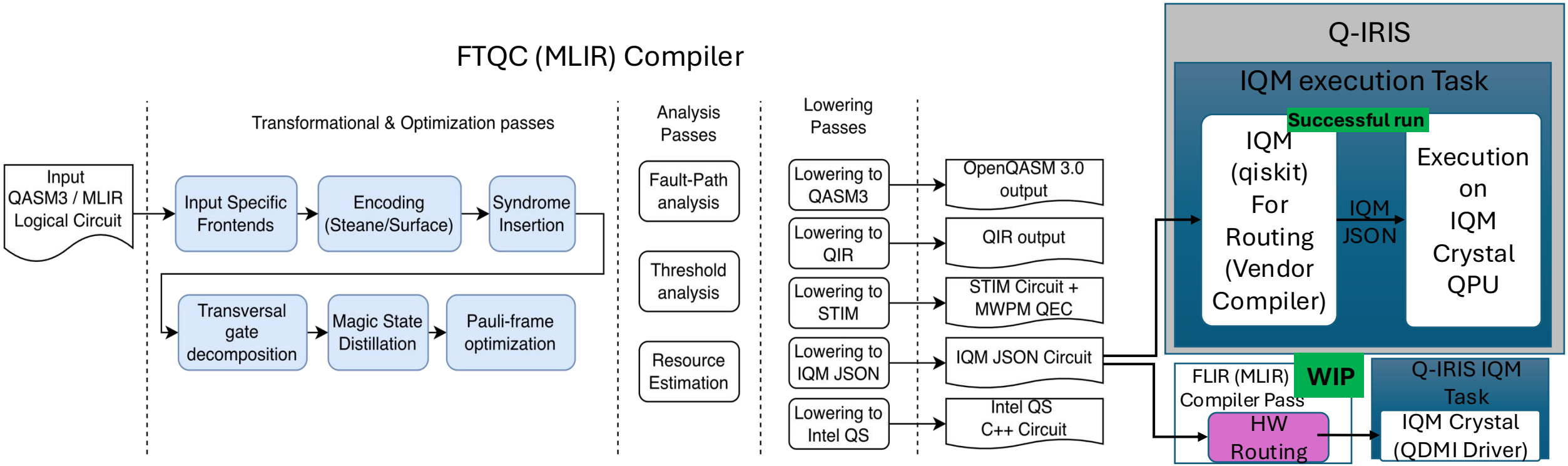
HIP: Heterogeneous compute Interface for Portability
CUDA: Compute Unified Device Architecture

- Supports a **QIR Kernel through QIR-EE runtime.**
- IQM is supported through QDMI
- Tasks can be expressed in Python, Julia, C and C++.
- Q-IRIS preserves the IRIS runtime task-based execution semantics.
- Quantum tasks can be on a task graph along with classical tasks.
- At runtime, quantum tasks are scheduled to run through QIR-EE or QDMI IQM RT.

* Q-IRIS developed under MACH-Q project but extended it with IQM support and uses QDMI driver under QSC

Miniskar et.al., "Classic and Quantum Task-Based Intelligent Runtime for QIRs Running on Multiple QPUs." In *2026 IEEE 19th Dallas Circuits and Systems Conference (DCAS)*, 2026
Miniskar et.al., "Q-IRIS: The Evolution of the IRIS Task-Based Runtime to Enable Classical-Quantum Workflows." In HPC Asia 2026 WS

Q-IRIS based Runtime to run FTQC driven physical circuits on IQM Crystal QPU



WIP: Work in Progress

* Q-IRIS developed under MACH-Q project but extended it with IQM support and uses QDMI driver

Thank you